

Reviewer Pack — Minimal Rank of Cocomplexes Bounds Resolution Refutation Siz...

Entry e7768ad485fd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 17:27:58 UTC

Minimal Rank of Cocomplexes Bounds Resolution Refutation Size

Entry ID: e7768ad485fd **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 17:27:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Cocomplexes) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

[‘For a Boolean circuit C with n inputs, the minimal rank of its cocomplex, denoted as $r_cocomplex(C)$, satisfies the inequality $\log_2(t(C)) \geq \Omega(r_cocomplex(C))$, where $t(C)$ is the minimum resolution refutation size for C .’, ‘This implies that circuits with lower cocomplex ranks require longer Resolution proofs.’, ‘For all circuits C , if $r_cocomplex(C) = O(1)$, then $t^*(C) = O(n^2)$.’]

Rationale (proposer’s reasoning):

[‘The cocomplex of a circuit encodes its complexity from an algebraic topology perspective, potentially revealing hidden structures that are not apparent through standard complexity measures.’, ‘Cocomplexes have been studied in algebraic topology, but their application to resolution proof length is novel and could provide new insights into the complexity of Boolean circuits.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 22f389f1543c46f6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across all n -input Boolean circuits C ($n \in [1, 40]$), the Spearman rank correlation coefficient between $\log_2(t^*(C))$ and $r_cocomplex(C)$ over 30 random seeds is ≥ 0.8 , with no seed producing a correlation coefficient < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - cocomplexes AND resolution proof complexity - algebraic topology co-complexes AND resolution refutation size - minimal rank cocomplexes AND bounds resolution proof

Top relevant hits considered: - [http://arxiv.org/abs/2206.00780v2] The BRST Double Complex for the Coupling of Gravity to Gauge Theories - [http://arxiv.org/abs/1304.6231v1] A-infinity Algebras Derived from Associative Algebras with a Non-Derivation Differential - [http://arxiv.org/abs/1802.08117v2] Topological spaces of persistence modules and their properties - [http://arxiv.org/abs/math/0502195v2] Hopf algebra structure on topological Hochschild homology - [http://arxiv.org/abs/1111.0731v2] On Hawaiian Groups of Some Topological Spaces - [http://arxiv.org/abs/2311.01864v1] SortNet: Learning To Rank By a Neural-Based Sorting Algorithm - [http://arxiv.org/abs/2106.15012v3] Level-rank duality of knot and link invariants - [http://arxiv.org/abs/1901.11354v2] The monic rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(1, 40)
    instances_tested = 30
    metric_values = []

    for _ in range(instances_tested):
        # Generate a random Boolean circuit C with n inputs
        C = [random.choice([0, 1]) for _ in range(2**n)]

        # Compute the cocomplex of C (placeholder implementation)
        # This is a placeholder and should be replaced with an actual algorithm
        r_cocomplex_C = random.randint(1, n)

        # Determine the minimum resolution refutation size t*(C) for each circuit
        # This is a placeholder and should be replaced with an actual algorithm
```

```

t_star_C = random.randint(1, 2**n)

# Measure the Spearman rank correlation between log2(t*(C)) and r_cocomplex(C)
if t_star_C <= 0 or r_cocomplex_C == 0:
    continue

metric_values.append((math.log2(t_star_C), r_cocomplex_C))

if not metric_values:
    return {
        "metric_name": "Spearman rank correlation",
        "metric_value": None,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

# Calculate Spearman rank correlation
n = len(metric_values)
ranks = {}
for i, (x, y) in enumerate(metric_values):
    if x not in ranks:
        ranks[x] = [i]
    else:
        ranks[x].append(i)
    if y not in ranks:
        ranks[y] = [i]
    else:
        ranks[y].append(i)

sorted_x = sorted(ranks.keys())
sorted_y = sorted(ranks.keys())
rank_x = {x: (sum(sorted_x.index(x) for x in xs) + len(xs) - 1) / (2 * len(xs)) for x, xs in ranks.items()}
rank_y = {y: (sum(sorted_y.index(y) for y in ys) + len(ys) - 1) / (2 * len(ys)) for y, ys in ranks.items()}

sum_dif_sq = sum((rank_x[x] - rank_y[y])**2 for x, y in metric_values)
spearman_corr = 1 - (6 * sum_dif_sq) / (n * (n**2 - 1))

return {
    "metric_name": "Spearman rank correlation",
    "metric_value": spearman_corr,
    "instances_tested": instances_tested,
    "conjecture_holds": spearman_corr >= 0.8 and all(spearman_corr >= 0.5 for _, y in metric_values),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}}}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)

```

```

std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_value"] < 0.5) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(r["metric_value"] < 0.5 for r in results):
    first_failing_seed = next(r["seed"] for r in results if r["metric_value"] < 0.5)
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11173
2	propose	ollama_remote	glm4:latest	0	0	14871
3	propose	ollama_remote	glm4:latest	0	0	10210
4	preregistration	ollama_remote	glm4:latest	0	0	6397
5	novelty	ollama_remote	glm4:latest	0	0	4552
6	novelty	ollama_remote	glm4:latest	0	0	6474
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19346
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11027
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14196
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12095
11	judge	ollama_remote	glm4:latest	0	0	24404

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134744 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in [research/audit/e7768ad485fd.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e7768ad485fd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e7768ad485fd.tar.gz` (if generated)